

Zur Verfügung stellen von Dateien in mobilen Betriebssystemen

Stephan Epp

25. März 2026

Inhaltsverzeichnis

1	Einleitung und Motivation	4
1.1	Definitionen	4
2	Technische Architektur mobiler Betriebssysteme	5
2.1	Das Sandboxing-Modell	5
2.2	Android: Speicherbereiche im Detail	5
2.3	iOS: Das App-Container-Modell	6
2.4	Die Rolle des MediaStore (Android)	6
3	Das Bereitstellungsprinzip — Formale Herleitung	7
3.1	Grundlegende Definitionen	7
3.2	Das Bereitstellungsprinzip als formaler Grundsatz	7
3.3	Herleitung aus dem Benutzersouveränitätsprinzip	8
4	Anwendungsklassen und ihre Bereitstellungspflicht	8
4.1	Die Kamera-Applikation	8
4.1.1	Korrekte Implementierung unter Android	8
4.1.2	Verletzungsfall: Privatspeicherung	9
4.2	Dokumenteneditoren	9
4.2.1	Storage Access Framework (SAF)	9
4.3	Audio-Anwendungen: Recorder und Downloader	10
4.3.1	Audio-Recorder	10
4.3.2	MP3-Downloader	10
4.4	Videobearbeitungsanwendungen	11
4.5	QR-Code- und Barcode-Generatoren	11
4.6	Zusammenfassung der Anwendungsklassen	11
5	Konsequenzen des Bereitstellungsprinzips	11
5.1	Konsequenzen für den Benutzer	11
5.1.1	Datensouveränität und Kontrolle	11
5.1.2	Datenpersistenz und Langzeitspeicherung	12
5.2	Konsequenzen für die Softwareentwicklung	12
5.2.1	API-Wahl und Architekturentscheidungen	12

5.2.2	Berechtigungssystem und Datenschutz	13
5.2.3	FileProvider für App-zu-App-Kommunikation	13
5.3	Konsequenzen für die Systemintegrität	14
5.3.1	Medianscan und Indexierung	14
5.3.2	Speicherbereinigung und Lifecycle	14
5.4	Konsequenzen für Sicherheit und Datenschutz	15
5.4.1	Das Paradox der Sandbox	15
5.4.2	DSGVO und Datensouveränität	15
5.4.3	Sicherheit: Zugriffssteuerung durch das Betriebssystem	15
5.5	Konsequenzen für Interoperabilität und Ökosystem	15
5.5.1	App-übergreifende Workflows	15
5.5.2	Cloud-Integration und Synchronisierung	16
5.5.3	Barrierefreiheit und Assistivtechnologien	16
5.6	Konsequenzen für Backup und Gerätewechsel	16
5.6.1	Systemseitiges Backup	16
5.6.2	Gerätewechsel und Migration	16
5.7	Konsequenzen für das App-Ökosystem und den Wettbewerb	17
5.7.1	Gegen Vendor Lock-in auf Dateiebene	17
5.7.2	Kompatibilität mit App-Store-Richtlinien	17
5.8	Konsequenzen für Fehlerdiagnose und Debugging	17
5.8.1	Auffindbarkeit von Fehlern	17
5.8.2	Testbarkeit	17
5.9	Konsequenzen für spezielle Randgebiete	18
5.9.1	IoT und verbundene Geräte	18
5.9.2	Medizinische Anwendungen	18
5.9.3	Unternehmenslösungen und MDM	18
5.9.4	Offline-First-Anwendungen	18
5.9.5	Progressive Web Apps (PWA)	18
6	Systemisches Modell der Dateibereitstellung	19
6.1	Das vollständige Bereitstellungsmodell	19
6.2	Qualitätsstufen der Bereitstellung	19
7	Empfehlungen und Best Practices	20
7.1	Für Entwickler	20
7.2	Für Plattformbetreiber	20
7.3	Für Benutzer	20
8	Zusammenfassung	21
8.1	Ausblick	21
8.1.1	Weiterentwicklung der Plattform-APIs und Betriebssystemarchitekturen	21
8.1.2	Das Origin Private File System und webbasierte Speicherparadigmen	22
8.1.3	Regulatorische und rechtliche Entwicklungen	22
8.1.4	Technologische Trends: Cloud-native Storage und Hybridarchitekturen	23
8.1.5	Künstliche Intelligenz und semantische Dateiverwaltung	24
8.1.6	Sicherheit und datenschutzfreundliche Bereitstellungsarchitekturen	24
8.1.7	IoT, Edge Computing und erweiterte Geräteklassen	25
8.1.8	Standardisierung und internationale Interoperabilität	26

8.1.9	Offene Forschungsfragen	26
8.1.10	Langfristige Vision: Das benutzerzentristische Dateisystem	27
	Literaturverzeichnis	27

1 Einleitung und Motivation

Mobile Betriebssysteme — allen voran Android und iOS — haben im vergangenen Jahrzehnt eine Architektur herausgebildet, die den Umgang mit Dateien fundamental von klassischen Desktop-Paradigmen unterscheidet. Während auf traditionellen Computersystemen eine Datei im Dateisystem des Betriebssystems als primärer Speicherort für persistente Daten gilt und Anwendungen lediglich lesen und schreiben, wurden auf mobilen Plattformen gezielt Sandboxing-Mechanismen eingeführt, die jede Anwendung in einem isolierten Speicherbereich kapseln. Diese Isolierung dient primär der Sicherheit und dem Datenschutz, erzeugt jedoch zugleich ein strukturelles Spannungsverhältnis: Anwendungen, die Dateien produzieren — Kameras, Dokumenteneditoren, Audio-Recorder, Mediaplayer — müssen entscheiden, *wo* diese Dateien residieren und *wer* auf sie zugreifen kann.

In der Praxis hat sich ein klares und ethisch wie technisch gebotenes Prinzip herausgestellt:

Das Bereitstellungsprinzip: Eine dateiproduzierende mobile Anwendung hat die erzeugten Dateien dem systemweiten Dateiverwaltungssystem des Betriebssystems *zur Verfügung zu stellen*. Dateien dürfen nicht dauerhaft im App-eigenen, für andere Anwendungen und den Benutzer unzugänglichen Speicher verbleiben.

Dieses Prinzip bildet den Kern der vorliegenden Arbeit. Es wird zunächst die technische Architektur mobiler Betriebssysteme untersucht (section 2), dann das Bereitstellungsprinzip formal definiert und hergeleitet (section 3), anschließend an konkreten Anwendungsklassen exemplifiziert (section 4), und schließlich werden alle Randgebiete und Konsequenzen umfassend diskutiert (section 5).

1.1 Definitionen

Der Begriff „Freigabe“ entstammt dem Sprachgebrauch der Praxis und ist historisch durch die Erfahrung im Umgang mit mobilen Apps entstanden. Er ist berechtigt, weil er das subjektive Erleben des Benutzers akkurat beschreibt: Eine Datei, die aus einer App heraus „freigegeben“ wird, ist plötzlich systemweit zugänglich. Dennoch ist er aus technischer Perspektive ungenau, da er eine vorherige Sperrung impliziert, die in der korrekten Implementierung gar nicht erst existieren sollte.

Definition 1.1 (Zur-Verfügung-Stellen). *Das Zur-Verfügung-Stellen einer Datei bezeichnet den Prozess, durch den eine dateiproduzierende Anwendung eine erzeugte Datei in einem für das Betriebssystem und autorisierte Drittanwendungen zugänglichen Speicherbereich ablegt und diesen Vorgang dem Medienspeicher-System des Betriebssystems signalisiert.*

Bemerkung 1.2. *Die Terminologie „zur Verfügung stellen“ ist „Freigabe“ vorzuziehen, da sie keine nachträgliche Handlung impliziert, sondern den Normalzustand beschreibt: Eine Datei, die ein Produktionsprozess erzeugt, ist von Natur aus dem System zur Verfügung zu stellen. Dieses Dokument verwendet beide Begriffe äquivalent, bevorzugt jedoch die präzisere Formulierung.*

2 Technische Architektur mobiler Betriebssysteme

2.1 Das Sandboxing-Modell

Moderne mobile Betriebssysteme basieren auf dem Prinzip der *Anwendungsisolation*. Jede installierte App erhält einen eigenen, abgeschotteten Speicherbereich, auf den ausschließlich diese App zugreifen kann. Dieses Modell, bekannt als *Sandboxing*, hat zwei wesentliche Ausprägungen:

1. **App-privater Speicher (Internal Storage):** Verzeichnisse wie `/data/data/<package>` (Android) oder `Documents/` (iOS-App-Container), auf die ausschließlich die eigene App zugreifen kann.
2. **Geteilter / systemweiter Speicher (Shared Storage):** Bereiche wie Galerien, Download-Ordner, Musikbibliotheken — zugänglich für Betriebssystem, Datei-Manager und anderen autorisierten Apps.

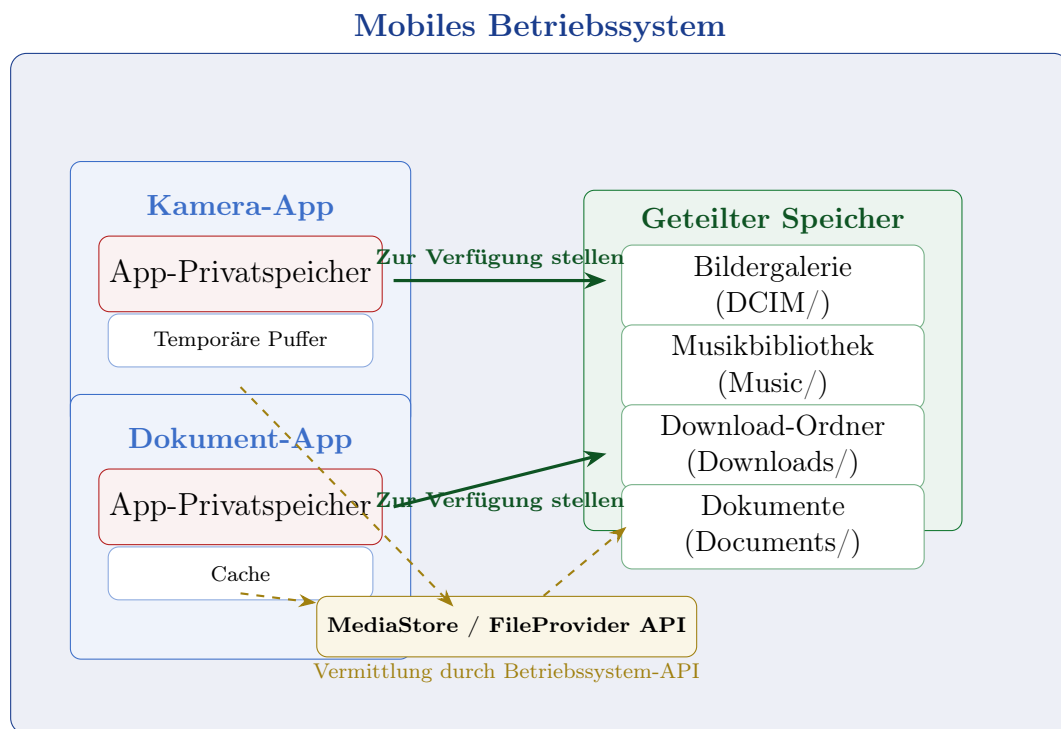


Abbildung 1: Schematische Darstellung der Speicherarchitektur eines mobilen Betriebssystems. App-private Bereiche (rot) sind von außen nicht zugänglich; der geteilte Speicher (grün) ist systemweit verfügbar. Das Zur-Verfügung-Stellen (grüne Pfeile) ist der zentrale Vorgang.

2.2 Android: Speicherbereiche im Detail

Unter Android gliedert sich der Speicher seit API-Level 29 (Android 10) durch *Scoped Storage* in klar definierte Bereiche:

Bereich	Pfad (typisch)	Zugänglichkeit	App-spezifisch
Internal Storage	/data/data/pkg/	Nur eigene App	Ja
External App-Dir	/sdcard/Android/data/	Nur eigene App + Root	Ja
Shared DCIM	/sdcard/DCIM/	Alle Apps mit Berechtigung	Nein
Shared Music	/sdcard/Music/	Alle Apps mit Berechtigung	Nein
Shared Downloads	/sdcard/Download/	Alle Apps mit Berechtigung	Nein
Shared Documents	/sdcard/Documents/	Alle Apps mit Berechtigung	Nein

Tabelle 1: Speicherbereiche unter Android mit Zugänglichkeit

2.3 iOS: Das App-Container-Modell

Unter iOS existiert ein ähnliches, wenngleich anders benanntes System. Jede App besitzt einen *App-Container*, der folgende Unterverzeichnisse enthält: **Documents/** (app-privat, im iCloud-Backup), **Library/** (Konfigurationen), **tmp/** (temporär). Systemweite Zugänglichkeit wird über das *Files-App-Framework* und die *Photos Library API* realisiert. Eine dateiproduzierende App muss auf iOS explizit Dateien über diese APIs dem System zur Verfügung stellen.

2.4 Die Rolle des MediaStore (Android)

Der **MediaStore** ist die zentrale Datenbank unter Android, die alle medienrelevanten Dateien (Bilder, Videos, Audio, Dokumente) katalogisiert. Er fungiert als Vermittler zwischen datenproduzierenden Apps und dem Rest des Systems:

```

1 // Korrekte Implementierung: Bild wird dem System zur Verfuegung
  gestellt
2 ContentValues values = new ContentValues();
3 values.put(MediaStore.Images.Media.DISPLAY_NAME, "foto_2026.jpg");
4 values.put(MediaStore.Images.Media.MIME_TYPE, "image/jpeg");
5 values.put(MediaStore.Images.Media.RELATIVE_PATH,
6             Environment.DIRECTORY_PICTURES + "/MeineApp");
7
8 Uri imageUri = getContentResolver().insert(
9     MediaStore.Images.Media.EXTERNAL_CONTENT_URI, values);
10
11 // Datei in den systemweiten Speicher schreiben
12 try (OutputStream out = getContentResolver().openOutputStream(imageUri))
13     {
14         bitmap.compress(Bitmap.CompressFormat.JPEG, 95, out);
15     } // Datei ist nun systemweit zugaenglich -- kein Scan noetig

```

Listing 1: Korrekte Uebergabe eines Bildes an den MediaStore unter Android

3 Das Bereitstellungsprinzip — Formale Herleitung

3.1 Grundlegende Definitionen

Definition 3.1 (Dateiproduzierende Anwendung). *Eine dateiproduzierende Anwendung A ist eine mobile Softwareanwendung, deren primärer Zweck oder ein wesentlicher Teilzweck darin besteht, persistente digitale Dateien zu erzeugen. Formal: A ist dateiproduzierend, wenn $\exists t : \text{Output}(A, t) \cap \mathcal{F} \neq \emptyset$, wobei $\mathcal{F}$ die Menge aller möglichen Dateitypen bezeichnet und t einen Ausführungszeitpunkt darstellt.*

Definition 3.2 (App-privater Speicher). *Der App-private Speicher $S_{\text{priv}}(A)$ einer Anwendung A ist der Speicherbereich, auf den ausschließlich A selbst (und privilegierte Systemprozesse) lesend und schreibend zugreifen können. Es gilt:*

$$\forall B \neq A : \text{Access}(B, S_{\text{priv}}(A)) = \perp$$

Definition 3.3 (Systemweiter Speicher). *Der systemweite Speicher S_{sys} ist der Speicherbereich, auf den das Betriebssystem, autorisierte Drittanwendungen und der Benutzer über Systemwerkzeuge zugreifen können:*

$$\forall B \in \mathcal{A}_{\text{auth}} : \text{Access}(B, S_{\text{sys}}) = \top$$

wobei $\mathcal{A}_{\text{auth}}$ die Menge der berechtigten Anwendungen ist.

Definition 3.4 (Bereitstellungsoperation). *Die Bereitstellungsoperation $\Phi : \mathcal{F} \times S_{\text{priv}} \rightarrow S_{\text{sys}}$ ist die Abbildung, die eine Datei $f \in \mathcal{F}$ aus dem App-privaten in den systemweiten Speicher überführt und den Systemmedienspeicher über die neue Datei informiert:*

$$\Phi(f) := (\text{Move}(f, S_{\text{sys}}), \text{Notify}(\text{MediaStore}, f))$$

3.2 Das Bereitstellungsprinzip als formaler Grundsatz

Grundsatz 3.5 (Das Bereitstellungsprinzip). *Sei A eine dateiproduzierende Anwendung und $f \in \mathcal{F}$ eine von A erzeugte Datei. Dann gilt unter allen Umständen:*

$$\text{Persist}(f) \Rightarrow \Phi(f)$$

*Das heißt: Jede Datei, die persistent gespeichert werden soll, **muss** durch die Bereitstellungsoperation Φ in den systemweiten Speicher überführt werden. Die dauerhafte Ablage in $S_{\text{priv}}(A)$ ist unzulässig.*

Grundsatz 3.6 (Verletzungsfall). *Eine Anwendung A verletzt das Bereitstellungsprinzip, wenn:*

$$\exists f \in \mathcal{F} : \text{Persist}(f) \wedge \neg \Phi(f) \wedge f \in S_{\text{priv}}(A)$$

Dies stellt einen schwerwiegenden Implementierungsfehler dar, der Benutzerrechte, Datensouveränität und Systemintegrität verletzt.

3.3 Herleitung aus dem Benutzersouveränitätsprinzip

Das Bereitstellungsprinzip folgt unmittelbar aus dem übergeordneten *Benutzersouveränitätsprinzip*, das besagt: Der Benutzer eines mobilen Geräts ist der rechtmäßige Eigentümer aller auf seinem Gerät erzeugten Daten. Daraus folgt:

Satz 3.7 (Datensouveränität impliziert Bereitstellungspflicht). *Sei u der Benutzer eines mobilen Geräts D und A eine dateiproduzierende Anwendung auf D . Dann gilt:*

$$\text{Souverän}(u, D) \Rightarrow \text{Zugriffsrecht}(u, f) \quad \forall f \in \text{Erzeugt}(A)$$

Da $\text{Zugriffsrecht}(u, f)$ nur dann erfüllt ist, wenn $f \in S_{\text{sys}}$, folgt das Bereitstellungsprinzip (grundsatz 3.5) als notwendige Bedingung.

Beweis. Der Benutzer u kann auf $S_{\text{priv}}(A)$ ohne spezielle (und üblicherweise nicht vorhandene) Berechtigungen nicht zugreifen. Folglich ist $\text{Zugriffsrecht}(u, f) = \perp$ für $f \in S_{\text{priv}}(A)$. Dies steht im Widerspruch zu $\text{Souverän}(u, D)$. Daher muss $f \in S_{\text{sys}}$ gelten. $\square$

4 Anwendungsklassen und ihre Bereitstellungspflicht

4.1 Die Kamera-Applikation

Die Kamera-Applikation ist das paradigmatische Beispiel einer dateiproduzierenden App. Ihre Ausgabe — fotografische Bilder und Videoaufnahmen — sind Primärdaten, die unmittelbar nach ihrer Erzeugung dem System zur Verfügung gestellt werden müssen.

4.1.1 Korrekte Implementierung unter Android

```
1 public void saveImageToGallery(Bitmap bitmap, Context context) {
2     ContentValues values = new ContentValues();
3     values.put(MediaStore.Images.Media.DISPLAY_NAME,
4         "IMG_" + System.currentTimeMillis() + ".jpg");
5     values.put(MediaStore.Images.Media.MIME_TYPE, "image/jpeg");
6     values.put(MediaStore.Images.Media.DATE_ADDED,
7         System.currentTimeMillis() / 1000);
8     values.put(MediaStore.Images.Media.RELATIVE_PATH,
9         Environment.DIRECTORY_DCIM + "/Camera");
10    values.put(MediaStore.Images.Media.IS_PENDING, 1); // Schreibe-Lock
11
12    Uri uri = context.getContentResolver().insert(
13        MediaStore.Images.Media.EXTERNAL_CONTENT_URI, values);
14
15    try (OutputStream out = context.getContentResolver()
16        .openOutputStream(uri)) {
17        bitmap.compress(Bitmap.CompressFormat.JPEG, 95, out);
18        // Schreibe-Lock aufheben: Datei jetzt systemweit sichtbar
19        values.clear();
20        values.put(MediaStore.Images.Media.IS_PENDING, 0);
21        context.getContentResolver().update(uri, values, null, null);
22    } catch (IOException e) {
23        context.getContentResolver().delete(uri, null, null);
24        throw new RuntimeException("Fehler beim Speichern", e);
25    }
26 }
```


4.1.2 Verletzungsfall: Privatspeicherung

Anti-Pattern: Datei im App-Privatspeicher.

Die folgende Implementierung **verletzt** das Bereitstellungsprinzip:

```
// FALSCH: Bild wird im App-privaten Verzeichnis gespeichert
File privateDir = context.getFilesDir(); // App-privat!
File imageFile = new File(privateDir, "foto.jpg");
// Kein MediaStore-Eintrag, keine systemweite Sichtbarkeit
// Benutzer kann Datei nicht in Galerie sehen
// Datei geht bei App-Deinstallation verloren
```

4.2 Dokumenteneditoren

Textverarbeitungsprogramme, Tabellenkalkulatoren, PDF-Editoren und verwandte Anwendungstypen produzieren Dokumente, die in der Regel langfristig gespeichert und von anderen Anwendungen weiterverarbeitet werden sollen.

Beispiel — Dokumenten-App unter Android.

Eine Dokumenten-App, die eine PDF-Datei erzeugt, muss diese in `/sdcard/Documents/` (über `MediaStore.Files` oder `Storage Access Framework`) ablegen und den entsprechenden MIME-Typ `application/pdf` registrieren. Die Datei ist dann über jeden installierten Dateimanager und jede PDF-Leseanwendung zugänglich.

4.2.1 Storage Access Framework (SAF)

Unter Android bietet das *Storage Access Framework* einen standardisierten Mechanismus, um Dateien systemweit zur Verfügung zu stellen, ohne direkte Dateipfad-Abhängigkeiten zu erzeugen:

```
1 // Benutzer waehlt Speicherort (systemweiter Dialog)
2 Intent intent = new Intent(Intent.ACTION_CREATE_DOCUMENT);
3 intent.addCategory(Intent.CATEGORY_OPENABLE);
4 intent.setType("application/pdf");
5 intent.putExtra(Intent.EXTRA_TITLE, "bericht_2026.pdf");
6 startActivityForResult(intent, REQUEST_CODE_CREATE_DOC);
7
8 // Im Callback: Datei in den vom Benutzer gewaehlten Pfad schreiben
9 @Override
10 protected void onActivityResult(int requestCode, int result, Intent data
11 ) {
12     if (requestCode == REQUEST_CODE_CREATE_DOC
13         && result == RESULT_OK) {
14         Uri uri = data.getData(); // Systemweiter URI
15         try (OutputStream out = getContentResolver()
16             .openOutputStream(uri)) {
17             writePdfContent(out); // Dokument schreiben
18             // Datei ist nun systemweit verfuegbar
19         }
20     }
```

4.3 Audio-Anwendungen: Recorder und Downloader

4.3.1 Audio-Recorder

Aufnahme-Anwendungen erzeugen Audiodateien (WAV, MP3, AAC, FLAC). Diese müssen nach Abschluss der Aufnahme in der Musikbibliothek oder dem Download-Ordner des Systems registriert werden:

```
1 ContentValues values = new ContentValues();
2 values.put(MediaStore.Audio.Media.DISPLAY_NAME, "aufnahme.m4a");
3 values.put(MediaStore.Audio.Media.MIME_TYPE, "audio/mp4");
4 values.put(MediaStore.Audio.Media.RELATIVE_PATH,
5             Environment.DIRECTORY_MUSIC + "/Aufnahmen");
6 values.put(MediaStore.Audio.Media.IS_MUSIC, 1);
7
8 Uri audioUri = getContentResolver().insert(
9     MediaStore.Audio.Media.EXTERNAL_CONTENT_URI, values);
10 // Audiodatei schreiben und IS_PENDING auf 0 setzen
```

Listing 4: Audioaufnahme dem System bereitstellen

4.3.2 MP3-Downloader

Anwendungen, die Audiodateien aus dem Internet herunterladen, stellen einen wichtigen Grenzfall dar. Der Download-Prozess erzeugt eine Datei, die dem Benutzer dauerhaft zur Verfügung stehen soll:

- 1: Download initiieren:** Die Datei wird temporär in einen Arbeitsspeicher geladen.
- 2: Überprüfung:** Integrität und Lizenzstatus der Datei werden geprüft.
- 3: Bereitstellung:** Die abgeschlossene Datei wird über `MediaStore` oder `DownloadManager` in den systemweiten Speicher (`Downloads/` oder `Music/`) überführt.
- 4: Benachrichtigung:** Das Betriebssystem wird über die neue Datei informiert.

```
1 DownloadManager.Request request = new DownloadManager.Request(
2     Uri.parse("https://example.com/audio.mp3"))
3     .setTitle("Audiodatei")
4     .setDescription("Download laeuft...")
5     .setNotificationVisibility(
6         DownloadManager.Request.VISIBILITY_VISIBLE_NOTIFY_COMPLETED)
7     .setDestinationInExternalPublicDir(
8         Environment.DIRECTORY_MUSIC, "audiodatei.mp3") // Systemweit!
9     .setMimeType("audio/mpeg");
10
11 DownloadManager dm = (DownloadManager) getSystemService(DOWNLOAD_SERVICE);
12 long downloadId = dm.enqueue(request);
13 // Datei wird automatisch im systemweiten Music/-Ordner gespeichert
```

Listing 5: Systemkonformer Datei-Download per DownloadManager

4.4 Videobearbeitungsanwendungen

Videobearbeitungs-Apps exportieren fertige Videos in den systemweiten Medienspeicher. Zwischendateien (Schnittdaten, Projekte, Render-Cache) können im App-privaten Speicher verbleiben — das fertige Exportprodukt jedoch *muss* systemweit bereitgestellt werden.

4.5 QR-Code- und Barcode-Generatoren

Anwendungen, die Bilder (QR-Codes, Barcodes) erzeugen, müssen diese auf Wunsch des Benutzers in die Galerie speichern — nicht nur im App-internen Cache.

4.6 Zusammenfassung der Anwendungsklassen

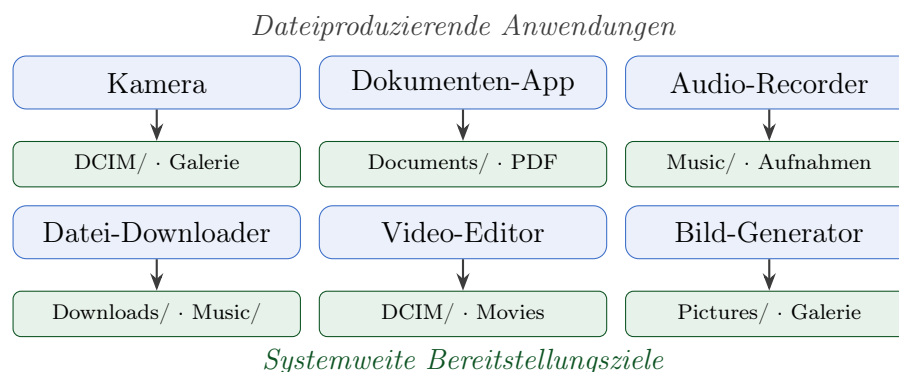


Abbildung 2: Übersicht der Anwendungsklassen und ihrer jeweiligen Bereitstellungsziele im systemweiten Speicher.

5 Konsequenzen des Bereitstellungsprinzips

5.1 Konsequenzen für den Benutzer

5.1.1 Datensouveränität und Kontrolle

Das Bereitstellungsprinzip ist die technische Umsetzung von Datensouveränität. Wenn eine App Dateien nur im App-privaten Speicher behält, verliert der Benutzer die effektive Kontrolle über seine eigenen Daten. Dies hat mehrere unmittelbare Folgen:

Benutzerauswirkungen bei Verletzung des Bereitstellungsprinzips:

- **Datenverlust bei Deinstallation:** App-private Daten werden bei Deinstallation unwiederbringlich gelöscht.
- **Unmöglichkeit des App-Wechsels:** Der Benutzer ist an die produzierende App gebunden, da Konkurrenzprodukte nicht auf die Daten zugreifen können.
- **Keine Backup-Integration:** Systemseitige Backups erfassen den App-privaten Speicher nur eingeschränkt oder gar nicht.
- **Keine Mediensuche:** Dateien sind in der systemweiten Galerie, im Dateimanager und in Suchfunktionen nicht auffindbar.

Konsequenz 5.1 (Benutzerautonomie). *Das Bereitstellungsprinzip ist eine notwendige Bedingung für Benutzerautonomie auf mobilen Geräten. Seine Verletzung erzeugt eine faktische Abhängigkeit (Vendor Lock-in) auf Dateiebene.*

5.1.2 Datenpersistenz und Langzeitspeicherung

Dateien im systemweiten Speicher überleben die deinstallation der erzeugenden App. Dies ist für den Benutzer grundlegend: Ein Foto bleibt auch dann erhalten, wenn die Kamera-App deinstalliert oder durch eine andere ersetzt wird.

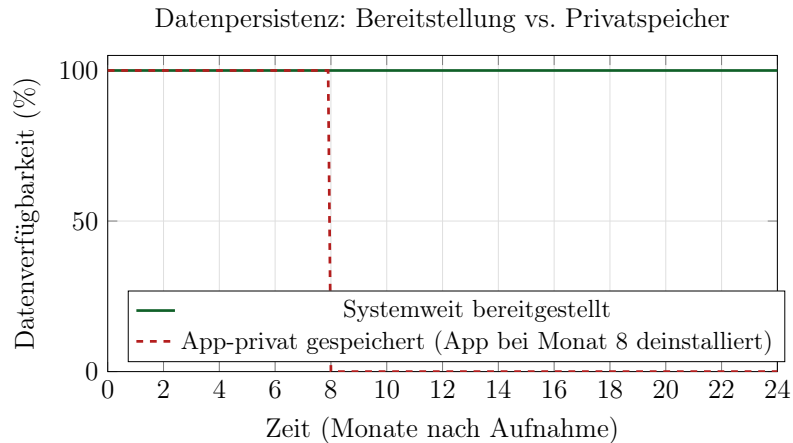


Abbildung 3: Datenpersistenz über die Zeit: Systemweit bereitgestellte Dateien sind dauerhaft verfügbar; App-privat gespeicherte Dateien gehen bei Deinstallation der App verloren.

5.2 Konsequenzen für die Softwareentwicklung

5.2.1 API-Wahl und Architekturentscheidungen

Entwickler müssen von Beginn an die korrekte API für Dateispeicherung wählen. Die Wahl des falschen APIs (z. B. `getFilesDir()` statt `MediaStore` für persistente Mediendateien) ist ein fundamentaler Architekturmangel, der nachträglich schwer zu korrigieren ist.

Satz 5.2 (API-Äquivalenz). *Für jede Datei f vom Typ $\tau \in \{\text{Bild}, \text{Video}, \text{Audio}, \text{Dokument}\}$ existiert eine eindeutige Ziel-API $API(\tau)$ im mobilen Betriebssystem, über die f korrekt dem System zur Verfügung zu stellen ist:*

$$API(\tau) = \begin{cases} \text{MediaStore.Images} & \text{falls } \tau = \text{Bild} \\ \text{MediaStore.Video} & \text{falls } \tau = \text{Video} \\ \text{MediaStore.Audio} & \text{falls } \tau = \text{Audio} \\ \text{MediaStore.Files} & \text{falls } \tau = \text{Dokument} \\ \text{DownloadManager} & \text{falls } \tau = \text{Sonstiges} \end{cases}$$

Dateityp	Korrekte (Android)	API (iOS)	Zielverzeichnis
Bild	MediaStore.Images	PHPhotoLibrary	DCIM/
Video	MediaStore.Video	PHPhotoLibrary	DCIM/
Audio	MediaStore.Audio	MPMediaLibrary	Music/
Dokument	MediaStore.Files / SAF	UIDocumentPicker	Documents/
Download	DownloadManager	URLSession + File- Manager	Downloads/

Tabelle 2: Korrekte APIs für verschiedene Dateitypen auf Android und iOS

5.2.2 Berechtigungssystem und Datenschutz

Das korrekte Zur-Verfügung-Stellen erfordert auf modernen Android-Versionen (API 33+) granulare Berechtigungen:

```

1 <manifest>
2   <!-- Schreiben in systemweite Medienbereiche (bis API 28) -->
3   <uses-permission android:name=
4     "android.permission.WRITE_EXTERNAL_STORAGE"
5     android:maxSdkVersion="28" />
6
7   <!-- Zugriff auf Bilder (API 33+) -->
8   <uses-permission android:name=
9     "android.permission.READ_MEDIA_IMAGES" />
10
11  <!-- Zugriff auf Videos (API 33+) -->
12  <uses-permission android:name=
13    "android.permission.READ_MEDIA_VIDEO" />
14
15  <!-- Zugriff auf Audio (API 33+) -->
16  <uses-permission android:name=
17    "android.permission.READ_MEDIA_AUDIO" />
18
19  <!-- Kamera-Berechtigung -->
20  <uses-permission android:name=
21    "android.permission.CAMERA" />
22 </manifest>

```

Listing 6: AndroidManifest.xml: Korrekte Berechtigungen fuer Medienzugriff

5.2.3 FileProvider für App-zu-App-Kommunikation

Wenn eine dateiproduzierende App eine Datei direkt an eine andere App übergeben möchte (z. B. „Teilen“ einer gerade erstellten Datei), ohne sie zunächst in den systemweiten Speicher zu schreiben, muss der FileProvider verwendet werden:

```

1 // FileProvider liefert einen Content-URI statt eines Dateipfads
2 File tempFile = new File(getCacheDir(), "temp_share.pdf");
3 // Datei erzeugen ...
4
5 Uri contentUri = FileProvider.getUriForFile(
6   this,

```

```

7      "de.epp.meinapp.fileprovider",
8      tempFile);
9
10 Intent shareIntent = new Intent(Intent.ACTION_SEND);
11 shareIntent.setType("application/pdf");
12 shareIntent.putExtra(Intent.EXTRA_STREAM, contentUri);
13 shareIntent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION);
14 startActivity(Intent.createChooser(shareIntent, "Teilen via"));

```

Listing 7: FileProvider fuer temporaere Dateieuebergabe

Bemerkung 5.3. *FileProvider ist für temporäre Übergeben geeignet; für dauerhafte systemweite Verfügbarkeit muss die Datei dennoch über MediaStore oder SAF bereitgestellt werden.*

5.3 Konsequenzen für die Systemintegrität

5.3.1 Medienscan und Indexierung

Das Betriebssystem führt regelmäßige Media-Scans durch, um den MediaStore aktuell zu halten. Wenn eine App Dateien ohne MediaStore-Eintrag in (theoretisch zugängliche) Bereiche schreibt, kann es zu Verzögerungen bei der Sichtbarkeit kommen. Die korrekte Implementierung nutzt MediaStore.insert() oder MediaScannerConnection.scanFile() unmittelbar nach dem Schreibvorgang.

```

1 // Fuer aeltere API-Level ohne automatischen MediaStore-Eintrag
2 MediaScannerConnection.scanFile(
3     context,
4     new String[]{ file.getAbsolutePath() },
5     new String[]{ "image/jpeg" },
6     (path, uri) -> {
7         // Scan abgeschlossen: Datei nun im MediaStore sichtbar
8         Log.d("MediaScan", "Datei verfuegbar unter: " + uri);
9     }
10 );

```

Listing 8: Expliziter MediaScan (Fallback, Legacy-Code)

5.3.2 Speicherbereinigung und Lifecycle

Das Betriebssystem kann unter Speicherdruck App-private Caches bereinigen, ohne den Benutzer zu informieren. Dateien im App-privaten cacheDir() sind *nicht persistent*. Dieser Mechanismus illustriert, warum dauerhafte Dateien *niemals* im App-privaten Speicher verbleiben dürfen.

Konsequenz 5.4 (Systemkohärenz). *Wird das Bereitstellungsprinzip flächendeckend eingehalten, entsteht ein kohärentes Dateisystem, in dem alle datenproduzierenden Aktivitäten des Benutzers an einem zentralen, systemweit zugänglichen Ort reflektiert werden. Dies ist eine Grundvoraussetzung für ein funktionstüchtiges und benutzerfreundliches Betriebssystem.*

5.4 Konsequenzen für Sicherheit und Datenschutz

5.4.1 Das Paradox der Sandbox

Ironischerweise kann die Sandboxing-Architektur zu Datenschutzverletzungen führen, wenn sie missbraucht wird: Eine App, die Dateien *nur* im App-privaten Speicher behält, entzieht diese Dateien der Kontrolle des Benutzers, obwohl das Sandboxing eigentlich dessen Schutz dienen soll.

Satz 5.5 (Sandbox-Paradox). *Die Sandboxing-Architektur mobiler Betriebssysteme, konzipiert als Schutzmaßnahme für den Benutzer, kann durch Nicht-Bereitstellung produzierter Dateien zu einer Einschränkung der Benutzerrechte pervertiert werden. Es gilt:*

$$\text{Sandbox}(A) + \neg\Phi(f) \Rightarrow \text{EffektiverDateinentzug}(u, f)$$

5.4.2 DSGVO und Datensouveränität

Im europäischen Rechtsraum ergibt sich eine datenschutzrechtliche Dimension: Die Datenschutz-Grundverordnung (DSGVO) garantiert Betroffenen unter anderem das Recht auf Datenzugang (Art. 15 DSGVO) und Datenübertragbarkeit (Art. 20 DSGVO). Eine App, die vom Benutzer produzierte Dateien im App-privaten Speicher einschließt, erschwert oder verhindert die Wahrnehmung dieser Rechte.

Rechtliche Konsequenz. Apps, die nutzergenerierte Dateien im App-privaten Speicher einschließen und dem Benutzer den Zugang zu seinen eigenen Daten systematisch verwehren, können gegen Art. 15 und Art. 20 DSGVO verstoßen, wenn diese Daten als personenbezogene Daten zu qualifizieren sind.

5.4.3 Sicherheit: Zugriffssteuerung durch das Betriebssystem

Der systemweite Speicher ist *nicht* unkontrolliert zugänglich. Unter Android müssen Apps seit API 23 Laufzeitberechtigungen für den Medienzugriff einholen; unter iOS gilt ein vergleichbares Modell. Das Betriebssystem stellt sicher, dass nur berechtigte Apps auf die Dateien zugreifen können — ein Sicherheitsgewinn gegenüber unkontrollierter App-privater Hortung.

5.5 Konsequenzen für Interoperabilität und Ökosystem

5.5.1 App-übergreifende Workflows

Wenn dateiproduzierende Apps das Bereitstellungsprinzip einhalten, ermöglichen sie komplexe, app-übergreifende Workflows. Beispiele:

1. **Foto-Workflow:** Kamera-App erzeugt Bild → Galerie zeigt es an → Bearbeitungs-App öffnet es → Teilen-Dialog übermittelt es → Cloud-Sync sichert es. Jeder dieser Schritte funktioniert nur, wenn das Bild systemweit verfügbar ist.
2. **Dokument-Workflow:** Dokumenten-App erzeugt PDF → E-Mail-App hängt es an → Drucker-App druckt es → Cloud-Speicher synchronisiert es.
3. **Audio-Workflow:** Recorder-App nimmt Sprache auf → Transkriptions-App verarbeitet die Audiodatei → Texteditor öffnet das Transkript.

5.5.2 Cloud-Integration und Synchronisierung

Systemweite Cloud-Sync-Dienste (Google Photos, iCloud, OneDrive) operieren auf dem systemweiten Dateisystem. Nur bereitgestellte Dateien werden automatisch synchronisiert und gesichert.

Konsequenz 5.6 (Cloud-Sync). *Das Bereitstellungsprinzip ist eine notwendige Bedingung für die automatische Cloud-Synchronisierung und damit für die Datensicherheit des Benutzers.*

5.5.3 Barrierefreiheit und Assistivtechnologien

Assistivtechnologien (Screenreader, Dokumenten-Vorlesedienste, Braille-Ausgabe) operieren auf systemweit zugänglichen Dateien. Eine App, die Dateien einschließt, schließt damit auch Benutzer mit Behinderungen von ihren eigenen Inhalten aus — ein klarer Verstoß gegen Barrierefreiheitsgrundsätze.

5.6 Konsequenzen für Backup und Gerätewechsel

5.6.1 Systemseitiges Backup

Sowohl Android (Auto Backup) als auch iOS (iCloud Backup) sichern den systemweiten Speicher vollständig. App-private Daten werden nur bei expliziter Konfiguration durch den App-Entwickler gesichert — und diese Konfiguration ist fehleranfällig und nicht standardisiert.

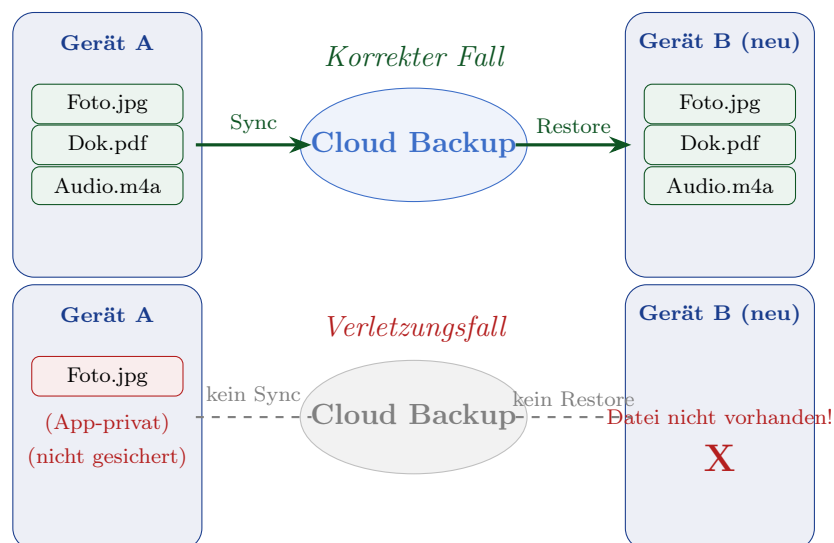


Abbildung 4: Backup und Gerätewechsel: Korrekt bereitgestellte Dateien werden gesichert und auf neuen Geräten wiederhergestellt; App-privat gespeicherte Dateien gehen verloren.

5.6.2 Gerätewechsel und Migration

Bei einem Gerätewechsel — oder beim Wechsel zwischen Android und iOS über Migrationswerkzeuge — sind ausschließlich systemweit verfügbare Dateien übertragbar. App-private Dateien können in der Regel nicht migriert werden, auch nicht durch Hersteller-Migrationswerkzeuge wie „Move to iOS“ oder „Google Backup & Restore“.

5.7 Konsequenzen für das App-Ökosystem und den Wettbewerb

5.7.1 Gegen Vendor Lock-in auf Dateiebene

Apps, die das Bereitstellungsprinzip verletzen, erzeugen einen unzulässigen *Vendor Lock-in*: Der Benutzer ist an die App gebunden, weil seine Daten in ihr „gefangen“ sind. Dies ist nicht nur ein technisches Problem, sondern ein wettbewerbspolitisches:

Konsequenz 5.7 (Wettbewerbsverzerrung). *Eine App, die Benutzerdaten im App-privaten Speicher einschließt, schafft künstliche Wechselbarrieren und verstößt gegen den Grundsatz des fairen Wettbewerbs im App-Ökosystem. Behörden im Bereich des Digitalmarktrechts (Digital Markets Act, EU) adressieren solche Praktiken zunehmend.*

5.7.2 Kompatibilität mit App-Store-Richtlinien

Sowohl der Google Play Store als auch der Apple App Store haben in ihren Entwickler-richtlinien Anforderungen formuliert, die implizit oder explizit das Bereitstellungsprinzip fordern:

- Google Play: *User Data Policy* verlangt, dass Apps mit nutzererzeugten Daten verantwortungsvoll umgehen.
- Apple App Store: *App Store Review Guidelines* (Abschnitt 5.1) verlangen, dass Apps den Datenzugang für Benutzer sicherstellen.

5.8 Konsequenzen für Fehlerdiagnose und Debugging

5.8.1 Auffindbarkeit von Fehlern

Wenn Dateien im App-privaten Speicher gespeichert werden, sind sie für Debugging-Werkzeuge (Android Debug Bridge, Xcode File Inspector) nur mit erhöhten Berechtigungen (root / dev-Modus) zugänglich. Dies erschwert die Fehlerdiagnose bei Produktionsproblemen.

5.8.2 Testbarkeit

Automationstests, die das Vorhandensein einer Datei nach einer App-Aktion prüfen, müssen auf den systemweiten Speicher zugreifen können. Bei korrekter Implementierung sind solche Tests einfach zu schreiben; bei App-privater Speicherung erfordern sie privilegierten Zugang.

```
1 @Test
2 public void testKamerafoto_wirdInGalerieBereitgestellt() {
3     // Foto aufnehmen
4     instrumentation.performCamera();
5
6     // Prüfen ob Datei im MediaStore vorhanden (systemweit)
7     String[] projection = {MediaStore.Images.Media._ID,
8                             MediaStore.Images.Media.DISPLAY_NAME};
9     Cursor cursor = context.getContentResolver().query(
10         MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
11         projection, null, null,
12         MediaStore.Images.Media.DATE_ADDED + " DESC");
13
14     assertNotNull("Kein MediaStore-Eintrag gefunden", cursor);
```

```

15     assertTrue("Galerie ist leer", cursor.moveToFirst());
16     // Test erfolgreich: Datei ist systemweit verfuegbar
17     cursor.close();
18 }

```

Listing 9: Automatisierter Test: Pruefung der systemweiten Bereitstellung

5.9 Konsequenzen für spezielle Randgebiete

5.9.1 IoT und verbundene Geräte

Wenn ein Smartphone über Bluetooth oder WiFi mit IoT-Geräten (Drohnen, Kameras, Smarthome-Sensoren) verbunden ist und Daten von diesen Geräten empfängt, gelten dieselben Bereitstellungspflichten. Eine Drohnen-App, die Luftaufnahmen herunterlädt, muss diese in der Galerie bereitstellen.

5.9.2 Medizinische Anwendungen

Medizinische Apps (EKG-Aufzeichnungen, Spirometrie-Daten, medizinische Bilder) stellen einen sensiblen Randbereich dar. Hier kollidieren:

- **Bereitstellungsprinzip:** Daten sollen dem Benutzer zugänglich sein.
- **Datenschutz:** Medizinische Daten sind hochsensibel (Art. 9 DSGVO).
- **Medizinproduktrecht:** Anforderungen an Datensicherheit und Integrität.

Die Lösung liegt nicht im Einschließen der Daten im App-privaten Speicher, sondern in verschlüsselter Speicherung im systemweiten Bereich mit benutzergesteuertem Zugriffsmanagement.

5.9.3 Unternehmenslösungen und MDM

Im Enterprise-Kontext mit Mobile Device Management (MDM) gelten spezifische Richtlinien. Hier kann eine kontrollierte Bereitstellung in dedizierte, unternehmensgesteuerte Bereiche (z. B. *Work Profile* unter Android) das Bereitstellungsprinzip unter gleichzeitiger Wahrung von Unternehmensrichtlinien realisieren.

5.9.4 Offline-First-Anwendungen

Apps, die primär offline arbeiten und Daten lokal zwischenspeichern (z. B. für spätere Synchronisierung), dürfen Arbeitsdaten temporär im App-privaten Cache halten. Sobald jedoch eine Datei als „fertiggestellt“ gilt, greift das Bereitstellungsprinzip uneingeschränkt.

5.9.5 Progressive Web Apps (PWA)

PWAs, die als mobile Apps installiert werden, unterliegen einem eingeschränkten Dateisystemzugang. Dennoch gilt das Bereitstellungsprinzip: Über die *File System Access API* (wo verfügbar) oder Download-Mechanismen müssen produzierte Dateien dem Benutzer systemweit zugänglich gemacht werden.

6 Systemisches Modell der Dateibereitstellung

6.1 Das vollständige Bereitstellungsmodell

Wir fassen alle Bestandteile zu einem vollständigen formalen Modell zusammen:

Definition 6.1 (Mobiles Dateisystem-Ökosystem). *Das mobile Dateisystem-Ökosystem $\mathcal{E}$ ist ein Tupel:*

$$\mathcal{E} = (\mathcal{A}, \mathcal{F}, S_{priv}, S_{sys}, \Phi, \mathcal{U}, \mathcal{P})$$

wobei:

$$\begin{aligned} \mathcal{A} &= \text{Menge aller installierten Anwendungen} \\ \mathcal{F} &= \text{Menge aller Dateitypen} \\ S_{priv} : \mathcal{A} &\rightarrow \mathcal{P}(\mathcal{F}) \quad (\text{App-private Speicherfunktion}) \\ S_{sys} &\in \mathcal{P}(\mathcal{F}) \quad (\text{Systemweiter Speicher}) \\ \Phi : \mathcal{F} \times \mathcal{A} &\rightarrow S_{sys} \quad (\text{Bereitstellungsoperation}) \\ \mathcal{U} &= \text{Benutzermenge} \\ \mathcal{P} : \mathcal{U} \times S_{sys} &\rightarrow \{\top, \perp\} \quad (\text{Zugriffsberechtigung}) \end{aligned}$$

Satz 6.2 (Konsistenz des Ökosystems). *Das Ökosystem $\mathcal{E}$ ist benutzerrechtskonform genau dann, wenn:*

$$\forall A \in \mathcal{A}_{prod}, \forall f \in \text{Erzeugt}(A), \forall u \in \mathcal{U} : \quad f \in S_{sys} \wedge \mathcal{P}(u, f) = \top$$

wobei $\mathcal{A}_{prod} \subseteq \mathcal{A}$ die Menge der dateiproduzierenden Anwendungen ist. Diese Bedingung ist äquivalent zum vollständigen Einhalten des Bereitstellungsprinzips durch alle Anwendungen in $\mathcal{A}_{prod}$.

6.2 Qualitätsstufen der Bereitstellung

Nicht alle Bereitstellungsimplementierungen sind gleich. Wir definieren ein Qualitätsstufenmodell:

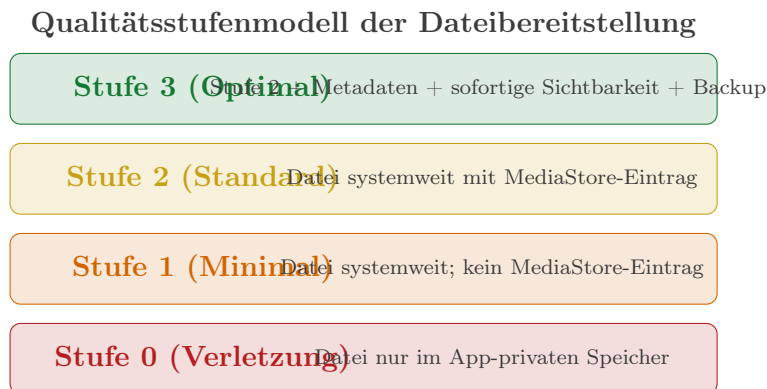


Abbildung 5: Qualitätsstufen der Dateibereitstellung. Nur Stufe 2 und 3 sind bereitstellungskonform; Stufe 3 ist die empfohlene Implementierung.

7 Empfehlungen und Best Practices

7.1 Für Entwickler

- E1. MediaStore als erste Wahl:** Für alle persistenten Mediendateien (Bilder, Videos, Audio) immer MediaStore verwenden, nie `getFilesDir()` oder `getExternalFilesDir()`.
- E2. SAF für Dokumente:** Das Storage Access Framework für nicht-Mediendateien verwenden, um maximale Benutzerfreiheit zu gewährleisten.
- E3. DownloadManager für Downloads:** Dateien aus dem Internet über DownloadManager herunterladen, der automatisch die systemweite Bereitstellung sicherstellt.
- E4. IS_PENDING-Muster verwenden:** Beim Schreiben großer Dateien das IS_PENDING-Flag nutzen, um Integrität sicherzustellen.
- E5. Sofortige Bereitstellung:** Die Bereitstellung unmittelbar nach Abschluss des Erzeugungsprozesses vornehmen, nicht verzögert oder auf Benutzerinteraktion wartend.
- E6. Aussagekräftige Metadaten:** Beim MediaStore-Eintrag vollständige Metadaten (Name, MIME-Typ, Datum, Beschreibung) eintragen.
- E7. Testen mit Integrationstest:** Automatisierte Tests schreiben, die die systemweite Verfügbarkeit von erzeugten Dateien prüfen.

7.2 Für Plattformbetreiber

- P1. Linting-Regeln:** Statische Analyse-Werkzeuge um Regeln erweitern, die die Verwendung von `getFilesDir()` für persistente Mediendateien als Warnung oder Fehler markieren.
- P2. App-Store-Reviews:** Systematische Prüfung auf Verletzungen des Bereitstellungsprinzips im App-Review-Prozess.
- P3. Entwicklerdokumentation:** Das Bereitstellungsprinzip explizit in Entwicklerdokumentationen als verpflichtend kennzeichnen.

7.3 Für Benutzer

- B1. Apps prüfen:** Überprüfen, ob erzeugte Dateien im Dateimanager sichtbar sind. Wenn nicht, ist die App wahrscheinlich bereitstellungswidrig.
- B2. Feedback geben:** Entwickler über fehlende systemweite Bereitstellung informieren (Store-Bewertung, Entwickler-Kontakt).
- B3. Alternativ-Apps wählen:** Bei systematischer Verletzung des Bereitstellungsprinzips auf konforme Alternativen wechseln.

8 Zusammenfassung

Die vorliegende Arbeit hat das *Bereitstellungsprinzip* als fundamentalen Grundsatz mobiler Softwarearchitektur hergeleitet und umfassend analysiert. Die wesentlichen Erkenntnisse lassen sich wie folgt zusammenfassen:

Kern-Erkenntnisse:

1. **Das Bereitstellungsprinzip ist unbedingt:** Jede dateiproduzierende App *muss* erzeugte Dateien dem systemweiten Speicher zur Verfügung stellen. Es gibt keine legitime Ausnahme für persistente Benutzerdaten.
2. **Es ist technisch umsetzbar:** Android und iOS bieten vollständige API-Unterstützung für korrekte Bereitstellung (MediaStore, SAF, PHPhotoLibrary).
3. **Es ist rechtlich geboten:** Datensouveränität, DSGVO und Wettbewerbsrecht stützen das Bereitstellungsprinzip.
4. **Die Terminologie ist zu präzisieren:** „Zur Verfügung stellen“ ist die korrekte Formulierung; sie beschreibt den Normalzustand, nicht eine nachträgliche Handlung.
5. **Die Konsequenzen sind weitreichend:** Backup, Interoperabilität, Barrierefreiheit, Cloud-Sync, Gerätewechsel, Wettbewerb, Rechtskonformität — all diese Bereiche hängen von der Einhaltung des Bereitstellungsprinzips ab.

Das Bereitstellungsprinzip ist keine optionale Qualitätseigenschaft, sondern ein architektonisches und ethisches Gebot. Mobile Betriebssysteme wurden konzipiert, um Benutzer zu befähigen — nicht, um Daten in App-Silos zu fragmentieren. Die Verantwortung für seine Einhaltung liegt bei Entwicklern, Plattformbetreibern und — durch ihre Marktmacht — bei den Benutzern selbst.

8.1 Ausblick

Die in dieser Arbeit dargelegten Grundsätze und Konsequenzen des Bereitstellungsprinzips sind keineswegs als abgeschlossenes Erkenntnisssystem zu verstehen. Die mobile Computing-Landschaft befindet sich in einem stetigen und teils rasanten Wandel, der sowohl technische als auch rechtliche und gesellschaftliche Dimensionen umfasst. Der folgende Ausblick skizziert die wichtigsten Entwicklungstendenzen, die das Bereitstellungsprinzip in den kommenden Jahren prägen, herausfordern und weiterentwickeln werden.

8.1.1 Weiterentwicklung der Plattform-APIs und Betriebssystemarchitekturen

Die Bereitstellungs-APIs beider großen mobilen Plattformen — Android und iOS — befinden sich in kontinuierlicher Weiterentwicklung. Auf Android-Seite wurde mit der Einführung von *Scoped Storage* in Android 10 ein grundlegender Paradigmenwechsel vollzogen: Apps erhalten keinen freien Dateisystemzugriff mehr, sondern operieren über streng typisierten APIs. Dieser Trend wird sich fortsetzen. Es ist zu erwarten, dass zukünftige Android-Versionen:

- die Granularität des Berechtigungssystems weiter verfeinern, etwa durch zeitlich begrenzte oder zweckgebundene Zugriffsberechtigungen auf Mediendateien;

- neue **MediaStore**-Kategorien einführen, die spezialisierte Dateitypen (medizinische Daten, Finanzdokumente, Bildungsmaterialien) semantisch erfassen;
- die *Health Connect*-Plattform als Modell für zukünftige, datentypspezifische Bereitstellungs-APIs etablieren — ähnlich wie HealthKit auf iOS einen standardisierten, sicheren Datenaustausch für Gesundheitsdaten ermöglicht;
- einen erweiterten *Photo Picker* bereitstellen, der Anwendungen den Zugriff auf Mediendateien ohne vollständige Lese-Berechtigungen ermöglicht und damit das Bereitstellungsprinzip schreibend von App-Seite konsequent einfordert.

Auf iOS-Seite ist eine sukzessive Öffnung des Dateisystems zu beobachten, die mit iOS 11 durch die Files-App und das `UIDocumentBrowserViewController`-Framework eingeleitet wurde. Zukünftige iOS-Versionen werden voraussichtlich die Inter-App-Kommunikation auf Dateiebene weiter erleichtern, das *Shared with You*-Framework ausbauen und die Abhängigkeit von proprietären iCloud-Strukturen reduzieren, indem offene Dateiformate und standardisierte Metadaten bevorzugt werden.

Ein besonders bedeutsamer Trend ist die zunehmende Konvergenz zwischen Desktop- und mobilen Betriebssystemen. Apples *Catalyst*-Technologie und Microsofts Integration von Android-Apps unter Windows (Windows Subsystem for Android) deuten darauf hin, dass die scharfe Grenze zwischen mobilen und Desktop-Dateisystemparadigmen langfristig verschwimmen könnte. Das Bereitstellungsprinzip wird dabei als Brückenkonzept fungieren müssen, das mobile Bereitstellungsmechanismen mit klassischen Desktop-Dateisystemzugriffen harmonisiert.

8.1.2 Das Origin Private File System und webbasierte Speicherparadigmen

Die Entstehung des *Origin Private File System* (OPFS) im Rahmen des *File System Access API*-Standards des W3C stellt eine fundamentale Herausforderung für das Bereitstellungsprinzip im Web-Kontext dar. OPFS bietet Progressive Web Apps einen performanten, privaten Dateispeicher — analog zum App-privaten Speicher nativer Apps. Gleichzeitig haben Progressive Web Apps über die `showSaveFilePicker()`-API die Möglichkeit, Dateien in den systemweiten Speicher zu schreiben.

Die Frage, welche dieser APIs für welchen Dateityp zu verwenden ist, spiegelt unmittelbar das Bereitstellungsprinzip wider: Temporäre Arbeitsdaten gehören in OPFS; persistente Benutzerdateien müssen über `showSaveFilePicker()` dem System zugänglich gemacht werden. Zukünftige Forschung sollte untersuchen, ob und wie das Bereitstellungsprinzip formal auf das Web-Ökosystem übertragen werden kann, insbesondere in Anbetracht der unterschiedlichen Sicherheitsmodelle zwischen Same-Origin-Policy und dem Android-Sandboxing-Ansatz.

8.1.3 Regulatorische und rechtliche Entwicklungen

Das rechtliche Umfeld für mobile Betriebssysteme und App-Ökosysteme befindet sich in einem historisch einzigartigen Wandel. Mehrere regulatorische Entwicklungen werden das Bereitstellungsprinzip direkt oder indirekt stärken.

Digital Markets Act (DMA). Der im November 2022 in Kraft getretene *Digital Markets Act* der Europäischen Union verpflichtet sogenannte *Gatekeeper* — zu denen

Apple und Google mit ihren App-Ökosystemen zählen — zu weitreichender Interoperabilität und Datenportabilität. Art. 6 Abs. 9 DMA verlangt insbesondere, dass Gatekeeper Nutzern den Zugang zu ihren generierten Daten ermöglichen. Dies deckt sich exakt mit dem Bereitstellungsprinzip: Daten, die auf einem Gerät erzeugt wurden, müssen dem Nutzer zugänglich sein. Zukünftige DMA-Durchsetzungsverfahren könnten explizit auf das Bereitstellungsprinzip als technischen Standard verweisen und damit seine Verbindlichkeit auf Plattformebene durchsetzen.

Data Act. Der im März 2024 in Kraft getretene *Data Act* (Verordnung (EU) 2023/2854) schafft horizontale Regeln für den Datenzugang und die Datennutzung in der gesamten EU-Wirtschaft. Für mobile Apps, die Nutzerdaten erzeugen, ergibt sich die Verpflichtung, diese Daten in interoperablen Formaten bereitzustellen. Das Bereitstellungsprinzip wird durch den Data Act eine explizite rechtliche Grundlage erhalten, die über die bisher aus der DSGVO mittelbar ableitbare Datenzugangsregelung hinausgeht.

AI Act und dateibasierte KI-Trainingsdaten. Die *Verordnung über Künstliche Intelligenz* (AI Act, Verordnung (EU) 2024/1689) führt neue Anforderungen an die Nachvollziehbarkeit von KI-Trainingsdaten ein. Wenn mobile Apps auf geräteerzeugten Dateien KI-Modelle trainieren oder diese Daten für systemische KI-Anwendungen verwenden, entsteht ein Spannungsfeld zwischen dem Bereitstellungsprinzip (Daten sollen systemweit zugänglich sein) und dem Schutz vor unerwünschter KI-Nutzung dieser Daten. Zukünftige Forschung muss untersuchen, wie das Bereitstellungsprinzip in KI-regulierten Umgebungen zu kalibrieren ist.

Internationaler Rechtsrahmen. Außerhalb der EU entstehen vergleichbare Regelungstendenzen: Der *American Data Privacy and Protection Act* (ADPPA), das *California Consumer Privacy Act* (CCPA) und Regelungen in Asien (Japans APPI, Indiens DPDP Act) konvergieren zunehmend auf gemeinsame Prinzipien der Datensouveränität, die das hier beschriebene Bereitstellungsprinzip international stützen.

8.1.4 Technologische Trends: Cloud-native Storage und Hybridarchitekturen

Die zunehmende Durchdringung mobiler Geräte mit permanenter Internetverbindung führt zu neuen Speicherparadigmen, die das Bereitstellungsprinzip in seiner klassischen, lokal orientierten Form herausfordern.

Cloud-as-Primary-Storage. Dienste wie Google Drive, iCloud Drive und OneDrive werden zunehmend nicht mehr als nachgelagerte Synchronisationsschicht, sondern als primärer Speicherort für erzeugte Dateien betrachtet. Das Bereitstellungsprinzip muss in diesem Kontext auf cloud-basierte Speicherziele ausgedehnt werden: Eine dateiproduzierende App, die Dateien direkt in der Cloud speichert, muss sicherstellen, dass diese Dateien über standardisierte Schnittstellen (WebDAV, CMIS, proprietäre Cloud-APIs) für den Benutzer zugänglich sind.

FUSE und virtuelle Dateisysteme. *Filesystem in Userspace* (FUSE) ermöglicht die transparente Integration von Remotspeichern in lokale Dateistrukturen. Zukünftige mobile Betriebssysteme könnten FUSE-artige Mechanismen einsetzen, um cloud-basierte und lokale Speicher unter einer einheitlichen Dateisystemabstraktion zusammenzuführen.

Das Bereitstellungsprinzip würde davon profitieren: Eine Datei, die in einem virtuellen Dateisystem abgelegt ist, wäre automatisch systemweit verfügbar, unabhängig von ihrem physischen Speicherort.

Peer-to-Peer-Dateisysteme. Dezentrale Dateisysteme wie *IPFS* (InterPlanetary File System) beginnen, in mobilen Anwendungsszenarien Relevanz zu erlangen. In solchen Systemen erfolgt die Bereitstellung nicht mehr über einen zentralen Systemdienst (MediaStore, Files-App), sondern über ein Netzwerkprotokoll. Das Bereitstellungsprinzip muss in diesem Fall auf die Frage erweitert werden, ob und wie eine Datei für den *Eigentümer* auffindbar und zugreifbar bleibt, auch wenn sie dezentral gespeichert ist.

8.1.5 Künstliche Intelligenz und semantische Dateiverwaltung

Vielleicht die tiefgreifendste Transformation des mobilen Dateisystems steht durch den Einsatz von KI-Methoden bevor. Aktuelle Entwicklungen lassen folgende Trends erkennen:

Automatische Dateikategorisierung. KI-gestützte Systeme (wie Googles *Gemini*-Integration in Android oder Apples *Intelligence*-Funktionen) sind zunehmend in der Lage, Dateiinhalte semantisch zu verstehen und automatisch zu kategorisieren. Zukünftig könnten mobile Betriebssysteme Dateien automatisch dem richtigen systemweiten Bereitstellungsziel zuordnen: Ein KI-Modell erkennt, dass eine erzeugte Datei ein Bild ist, und legt sie automatisch im korrekten MediaStore-Bucket ab, ohne dass die App explizit eine `insert()`-Operation aufrufen muss. Dies würde das technische Risiko von Bereitstellungsverletzungen aus Implementierungsfehlern erheblich reduzieren.

On-Device-AI und datenschutzwahrende Verarbeitung. Die Verlagerung von KI-Verarbeitung auf das Gerät selbst (On-Device-AI) erzeugt neue Arten dateiproduzierender Prozesse: KI-generierte Bilder, zusammengefasste Dokumente, transkribierte Audiodateien. Das Bereitstellungsprinzip gilt für diese Ausgaben uneingeschränkt — auch KI-erzeugte Inhalte müssen systemweit zugänglich gemacht werden, wenn der Benutzer sie als persistente Ergebnisse betrachtet.

Semantische Metadatenanreicherung. Zukünftige Bereitstellungssysteme werden über reine Dateipfade hinausgehen und Dateien mit semantischen Metadaten anreichern. Der MediaStore könnte um Felder für KI-erzeugte Beschreibungen, erkannte Objekte oder inhaltliche Zusammenfassungen erweitert werden. Das Bereitstellungsprinzip müsste in diesem Fall nicht nur die *physische* Verfügbarkeit einer Datei fordern, sondern auch ihre *semantische* Auffindbarkeit durch systemweite Suchfunktionen.

8.1.6 Sicherheit und datenschutzfreundliche Bereitstellungsarchitekturen

Das Spannungsverhältnis zwischen systemweiter Bereitstellung und Datenschutz wird in Zukunft durch neue technische Architekturen aufgelöst werden müssen.

Vertrauliche Bereitstellung. Eine vielversprechende Forschungsrichtung ist die *vertrauliche Bereitstellung*: Dateien werden systemweit bereitgestellt, jedoch in einer Weise verschlüsselt, die nur autorisierten Apps und Benutzern den Inhaltszugriff ermöglicht.

Technologien wie *Trusted Execution Environments* (TEE) auf ARM-Prozessoren (Trust-Zone) könnten als Basis für schlüsselverwaltende Bereitstellungsinfrastruktur dienen. In diesem Modell ist eine Datei *auffindbar* (Bereitstellungsprinzip erfüllt) und gleichzeitig *inhaltlich geschützt* (Datenschutz gewahrt).

Attributbasierte Zugangskontrolle. Klassische Zugangskontrolle basiert auf Identitäten (welche App darf zugreifen?). Zukünftige Systeme könnten *attributbasierte Zugangskontrolle* (ABAC) einsetzen: Eine Datei ist systemweit bereitgestellt, aber Zugriff wird nur Anwendungen gewährt, die bestimmte Attribute aufweisen (z. B. „ist als medizinische App zertifiziert“ oder „hat explizite Nutzerfreigabe für diesen Dateityp“). Dies würde das Bereitstellungsprinzip mit feingranularem Datenschutz verbinden.

Datensparsamkeit und typenspezifische Bereitstellung. Im Lichte von Art. 5 Abs. 1 lit. c DSGVO (Datensparsamkeit) entsteht die Frage, ob das Bereitstellungsprinzip zu einer systemweiten Nachverfolgbarkeit führen kann, die dem Datensparsamkeitsprinzip widerspricht. Konzeptuell ist zu betonen: Das Bereitstellungsprinzip fordert *Zugänglichkeit für den Benutzer*, nicht *Sichtbarkeit für Dritte*. Zukünftige Implementierungen müssen diese Unterscheidung technisch präziser abbilden.

8.1.7 IoT, Edge Computing und erweiterte Geräteklassen

Die Grenzen des „mobilen Geräts“ verschwimmen zunehmend. Smartwatches, AR/VR-Brillen, Connected Cars und smarte Haushaltsgeräte werden zu dateiproduzierenden Systemen, für die das Bereitstellungsprinzip Anwendung finden muss.

Wearables und Gesundheitsdaten. Smartwatch- und Fitness-Tracker-Apps erzeugen kontinuierlich Dateien (Schlafprotokolle, EKG-Aufzeichnungen, Bewegungsdaten). Die zugehörigen mobilen Apps müssen diese Daten über APIs wie Apple HealthKit (iOS) oder Android Health Connect systemweit bereitstellen. Die besondere Sensibilität dieser Daten (Art. 9 DSGVO: besondere Kategorien personenbezogener Daten) erfordert dabei ein differenzierteres Bereitstellungsmodell, das Zugänglichkeit und Schutz ausbalanciert.

AR/VR-Geräte und räumliche Daten. Augmented-Reality- und Virtual-Reality-Headsets (Apple Vision Pro, Meta Quest) erzeugen neuartige Dateiformate: räumliche Scans, 3D-Objekte, immersive Videoinhalte. Es ist zu erwarten, dass zukünftige **MediaStore**-Kategorien diese Dateitypen aufnehmen werden. Das Bereitstellungsprinzip postuliert: Auch 3D-Inhalte, die auf einem AR/VR-Gerät erzeugt werden, müssen über standardisierte APIs dem System zur Verfügung gestellt werden.

Vernetzte Fahrzeuge und mobile Arbeitsumgebungen. In moderne Kraftfahrzeuge integrierte Betriebssysteme (Android Automotive, CarPlay-unterstützte Systeme) bilden erweiterte mobile Plattformen. Dashcam-Aufnahmen, Navigationshistorien und fahrzeugintern erzeugte Dokumente fallen unter das Bereitstellungsprinzip. Zukünftige Fahrzeugbetriebssysteme werden einen eigenen Medienspeicher entwickeln müssen, der mit mobilen Gerätearchiven synchronisiert werden kann.

8.1.8 Standardisierung und internationale Interoperabilität

Ein wesentliches Desiderat der zukünftigen Forschung und Normierungsarbeit ist die plattformübergreifende Standardisierung des Bereitstellungsprinzips.

W3C-Standardisierung. Die *File System Access API* des W3C ist ein erster Schritt zur Standardisierung webbasierter Dateibereitstellung. Zukünftige W3C-Standards sollten explizit das Bereitstellungsprinzip kodieren und klare Anforderungen an das Schreiben persistenter Nutzerdaten in zugängliche Speicherbereiche formulieren. Das *Storage Foundation API* ist ein vielversprechender Anknüpfungspunkt für diese Standardisierungsarbeit.

ISO/IEC-Normierung. Im Bereich der mobilen Anwendungssicherheit existieren Normen wie ISO/IEC 27034 (Application Security). Eine Erweiterung dieser Normen um explizite Anforderungen an die Dateibereitstellung würde das Bereitstellungsprinzip in international anerkannte Qualitätsstandards integrieren.

Plattformübergreifende Interoperabilität. Das größte offene Problem ist die Interoperabilität zwischen Android und iOS. Eine auf iOS erstellte Datei kann nicht automatisch über den Android-MediaStore zugänglich gemacht werden und umgekehrt. Cloud-Synchronisationsdienste lösen dieses Problem partiell, setzen aber proprietäre Infrastruktur voraus. Eine zukünftige *Mobile File Provisioning Interoperability Specification* — vergleichbar mit dem *WebDAV*-Standard für Netzwerkdateisysteme — könnte plattformübergreifende Bereitstellung ohne Cloud-Abhängigkeit realisieren.

8.1.9 Offene Forschungsfragen

Die vorliegende Arbeit wirft eine Reihe von Forschungsfragen auf, die einer systematischen wissenschaftlichen Bearbeitung bedürfen:

- F1. Formale Verifikation:** Lässt sich das Bereitstellungsprinzip in formale App-Verifikationswerkzeuge integrieren? Können statische Analysatoren (z. B. auf Basis von *Datalog*-Regeln oder Typtheorie) Verletzungen des Bereitstellungsprinzips zur Compile-Zeit erkennen?
- F2. Automatisches Refactoring:** Können KI-gestützte Code-Transformatoren (z. B. GitHub Copilot oder ähnliche Systeme) Implementierungen, die gegen das Bereitstellungsprinzip verstoßen, automatisch identifizieren und korrekte Alternativen vorschlagen?
- F3. Empirische Prävalenz:** Wie verbreitet sind Verletzungen des Bereitstellungsprinzips in der realen App-Landschaft? Eine systematische Analyse über den Google Play Store und den Apple App Store — basierend auf statischer Analyse und Laufzeittests — könnte empirische Daten zu dieser Frage liefern.
- F4. Benutzerpräferenzen:** Welche Präferenzen haben Benutzer bezüglich der Zugänglichkeit ihrer erzeugten Dateien? Empirische Usability-Studien könnten das Bereitstellungsprinzip durch nutzerzentrierte Evidenz stützen oder differenzieren.
- F5. Performanz-Implikationen:** Hat die systemweite Bereitstellung von Dateien messbare Auswirkungen auf die App-Performanz (Latenz beim Schreiben, Energieverbrauch

durch MediaStore-Operationen)? Benchmarking-Studien könnten die praktischen Kosten der korrekten Implementierung quantifizieren.

- F6. Quantitativer Datenverlust:** Wie groß ist der tatsächliche Datenverlust, der durch Verletzungen des Bereitstellungsprinzips entsteht? Eine Befragungsstudie mit mobilen Nutzern, die Datenverluste nach App-Deinstallationen erlebt haben, könnte die gesellschaftlichen Kosten dieser Verletzungen sichtbar machen.
- F7. Semantisches Bereitstellungsmodell:** Ist ein erweitertes Bereitstellungsmodell denkbar, das nicht nur Dateien, sondern auch strukturierte Daten (Datenbanken, App-Zustände, Konfigurationen) dem Bereitstellungsprinzip unterwirft? Diese Erweiterung würde das Bereitstellungsprinzip zu einem umfassenden *Datensouveränitätsprinzip* für mobile Geräte verallgemeinern.

8.1.10 Langfristige Vision: Das benutzerzentristische Dateisystem

Auf einer übergeordneten Ebene zeigt die vorliegende Analyse, dass das Bereitstellungsprinzip Teil einer größeren Vision für mobile Betriebssysteme ist: das *benutzerzentristische Dateisystem*, in dem der Benutzer — nicht die App — als Eigentümer und primärer Verwalter aller erzeugten Daten fungiert.

Diese Vision impliziert einen strukturellen Wandel: Statt dass Apps entscheiden, *wo* und *wie* Dateien gespeichert werden, übernimmt das Betriebssystem die Verantwortung für eine optimale Bereitstellung, basierend auf Dateityp, Nutzungspräferenzen und rechtlichen Anforderungen. Apps werden zu reinen Produzenten; das Betriebssystem wird zum intelligenten Verteiler und Verwalter.

Technologische Bausteine für diese Vision sind bereits erkennbar: Androids *MediaStore* mit seinen typisierten Buckets, Apples *PHPhotoLibrary* mit ihrer nicht-destruktiven Bearbeitungsphilosophie, Googles *Personal Data Store*-Konzepte — all diese Entwicklungen deuten in dieselbe Richtung. Das Bereitstellungsprinzip ist damit nicht nur ein technischer Grundsatz der gegenwärtigen mobilen Softwareentwicklung, sondern auch ein Wegweiser für die Architektur zukünftiger mobiler Betriebssysteme.

Die konsequente Anwendung und Weiterentwicklung des Bereitstellungsprinzips wird langfristig dazu beitragen, das Vertrauen der Nutzer in mobile Plattformen zu stärken, digitale Souveränität zu fördern und eine fairere, interoperablere digitale Infrastruktur zu schaffen — eine Infrastruktur, in der Daten dem dienen, der sie erzeugt hat: dem Benutzer selbst.

Literatur

- [1] Google LLC. *Scoped Storage changes in Android 10*. Android Developers Documentation, 2019. <https://developer.android.com/about/versions/10/privacy/changes#scoped-storage>
- [2] Google LLC. *MediaStore API Reference*. Android Developers Documentation, 2024. <https://developer.android.com/reference/android/provider/MediaStore>
- [3] Google LLC. *Data and file storage overview*. Android Developers Documentation, 2024. <https://developer.android.com/training/data-storage>

- [4] Google LLC. *Open files using storage access framework*. Android Developers Documentation, 2024. <https://developer.android.com/guide/topics/providers/document-provider>
- [5] Google LLC. *FileProvider — Setting Up File Sharing*. Android Developers Documentation, 2024. <https://developer.android.com/reference/androidx/core/content/FileProvider>
- [6] Google LLC. *DownloadManager API Reference*. Android Developers Documentation, 2024. <https://developer.android.com/reference/android/app/DownloadManager>
- [7] Google LLC. *Request app permissions — Android Developers*. Android Developers Documentation, 2024. <https://developer.android.com/training/permissions/requesting>
- [8] Google LLC. *MediaScannerConnection — Android Developers*. Android Developers Documentation, 2024. <https://developer.android.com/reference/android/media/MediaScannerConnection>
- [9] Google LLC. *Back up user data with Auto Backup*. Android Developers Documentation, 2024. <https://developer.android.com/identity/data/autobackup>
- [10] Apple Inc. *PHPhotoLibrary — PhotoKit Framework Reference*. Apple Developer Documentation, 2024. <https://developer.apple.com/documentation/photokit/phphotolibrary>
- [11] Apple Inc. *File System Programming Guide for iOS*. Apple Developer Documentation, 2023. <https://developer.apple.com/library/archive/documentation/FileManagement/Conceptual/FileSystemProgrammingGuide/Introduction/Introduction.html>
- [12] Apple Inc. *UIDocumentBrowserViewController — Providing Access to Directories*. Apple Developer Documentation, 2024. <https://developer.apple.com/documentation/uikit/uidocumentbrowserviewController>
- [13] Apple Inc. *iCloud Storage and Backup Overview*. Apple Developer Documentation, 2024. <https://developer.apple.com/icloud/documentation/>
- [14] Europäisches Parlament und Rat der Europäischen Union. *Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates (Datenschutz-Grundverordnung)*. Amtsblatt der Europäischen Union, L 119, S. 1–88, 4. Mai 2016.
- [15] Europäisches Parlament und Rat der Europäischen Union. *Verordnung (EU) 2022/1925 über bestreithare und faire Märkte im digitalen Sektor (Digital Markets Act)*. Amtsblatt der Europäischen Union, L 265, S. 1–66, 12. Oktober 2022.
- [16] Europäisches Parlament und Rat der Europäischen Union. *Verordnung (EU) 2023/2854 über harmonisierte Vorschriften für einen fairen Datenzugang und eine faire Datennutzung (Data Act)*. Amtsblatt der Europäischen Union, L 2023/2854, 13. Dezember 2023.

- [17] Europäisches Parlament und Rat der Europäischen Union. *Verordnung (EU) 2024/1689 zur Festlegung harmonisierter Vorschriften für künstliche Intelligenz (Gesetz über künstliche Intelligenz)*. Amtsblatt der Europäischen Union, L 2024/1689, 12. Juli 2024.
- [18] Enck, W., Ongtang, M., und McDaniel, P. *On Lightweight Mobile Phone Application Certification*. In: *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS '09)*, S. 235–245. ACM, 2009.
- [19] Smalley, S., und Craig, R. *Security Enhanced (SE) Android: Bringing Flexible MAC to Android*. In: *Proceedings of the 20th Annual Network & Distributed System Security Symposium (NDSS 2013)*. Internet Society, 2013.
- [20] Felt, A. P., Ha, E., Egelman, S., Haney, A., Chin, E., und Wagner, D. *Android Permissions: User Attention, Comprehension, and Behavior*. In: *Proceedings of the 8th Symposium on Usable Privacy and Security (SOUPS 2012)*. ACM, 2012.
- [21] Zuboff, S. *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. PublicAffairs, New York, 2019.
- [22] Shapiro, C., und Varian, H. R. *Information Rules: A Strategic Guide to the Network Economy*. Harvard Business School Press, Boston, 1999.
- [23] W3C WHATWG. *File System Standard*. Living Standard, 2024. <https://fs.spec.whatwg.org/>
- [24] Benet, J. *IPFS — Content Addressed, Versioned, P2P File System*. arXiv:1407.3561, 2014. <https://arxiv.org/abs/1407.3561>
- [25] Pinto, S., und Santos, N. *Demystifying Arm TrustZone: A Comprehensive Survey*. *ACM Computing Surveys*, 51(6):130:1–130:36, 2019.
- [26] Hu, V. C., Ferraiolo, D., Kuhn, R., Friedman, A. R., Lang, A. J., Cogdell, M. M., Schnitzer, A., Sandlin, K., Miller, R., und Scarfone, K. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162. NIST, 2014.
- [27] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., und Adam, H. *Searching for MobileNetV3*. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2019)*, S. 1314–1324. IEEE, 2019.
- [28] Google LLC. *Health Connect — Android Developers*. Android Developers Documentation, 2024. <https://developer.android.com/health-and-fitness/guides/health-connect>
- [29] Europäische Kommission. *Digital Markets Act: Erster Bewertungsbericht zu den Gatekeeper-Verpflichtungen*. Europäische Kommission, Brüssel, 2024. https://commission.europa.eu/strategy-and-policy/priorities-2019-2024/europe-fit-digital-age/digital-markets-act-ensuring-fair-and-contestable-digital-markets_de

- [30] Tanenbaum, A. S., und Bos, H. *Modern Operating Systems*, 4. Auflage. Pearson, Upper Saddle River, NJ, 2015.
- [31] Google LLC. *Application Fundamentals — Android Developers*. Android Developers Documentation, 2024. <https://developer.android.com/guide/components/fundamentals>
- [32] Google LLC. *Content providers — Android Developers*. Android Developers Documentation, 2024. <https://developer.android.com/guide/topics/providers/content-providers>
- [33] World Wide Web Consortium (W3C). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 5. Oktober 2023. <https://www.w3.org/TR/WCAG22/>